SID34 - TD 1 et 2

Oumoukelthoum Sidenna

05/12/2025

Exercice 1
Soit (X, Y) un couple aléatoire à valeurs dans $\mathbb{R}^d \times \mathbb{R}$.

1. Rappeler la définition du risque quadratique en régression.
2. Montrer que la fonction de régression $m^\star(x) = \mathbb{E}[Y \mid X = x]$ minimise ce risque.

Exercice 2
Pour un point (x_i, y_i), écrire :

- La perte MSE.
- La perte hinge.

Expliquer pour chacune :

1. Le type d'erreur qu'elle pénalise le plus fortement.
2. Son impact sur les valeurs aberrantes.

Exercice 3
On considère un classifieur linéaire $f(x) = w^\top x + b$, avec $w^\top = (1, -1)$ et $b = 0$. On observe les trois points suivants :

$$x_1 = (2,1),\ y_1 = 1, \qquad x_2 = (1,3),\ y_2 = -1, \qquad x_3 = (0,2),\ y_3 = -1.$$

1. Calculer pour chaque point i la marge $m_i = y_i f(x_i)$.
2. Calculer pour chaque point les valeurs :

$$\ell_{\text{hinge}}(m_i) = \max(0,\, 1 - m_i), \quad \ell_{\log}(m_i) = \log(1 + e^{-m_i}), \quad \ell_{0-1}(m_i) = \mathbf{1}_{m_i < 0}.$$

3. Calculer le risque empirique pour chacune des trois pertes.
4. Classer les pertes selon le risque empirique obtenu. Que peut-on conclure pour ce classifieur ?

Vérification numérique

In [1]:
```python
import numpy as np

X = np.array([[2,1],[1,3],[0,2]])
y = np.array([1,-1,-1])

w = np.array([1,-1])
b = 0

margins = y*(X.dot(w) + b)
print("Margins:", margins)

hinge = np.maximum(0, 1-margins)
logistic = np.log(1+np.exp(-margins))
zero_one = (margins<0).astype(int)

print("Hinge R:", hinge.mean())
print("Logistic R:", logistic.mean())
print("0-1 R:", zero_one.mean())
```

Exercice 4 (Sélection de modèle et compromis biais–variance)
On étudie l'impact de la complexité du modèle sur l'erreur d'entraînement et l'erreur de test à l'aide de la régression polynomiale. On rappelle que l'on dispose d'un jeu d'entraînement $(x_{\text{train}}, y_{\text{train}})$ et d'un jeu de test $(x_{\text{test}}, y_{\text{test}})$ générés selon le modèle bruité

$$y = x^2 - x - 2 + \varepsilon.$$

On définit, pour chaque degré $k \in \{0, 1, 2, 3, 4, 5\}$, le modèle de régression polynomiale de degré k :

$$\hat{y}^{(k)}(x) = \theta_0 + \theta_1 x + \cdots + \theta_k x^k.$$

Les fonctions suivantes sont fournies dans le notebook :

- `make_design_matrix` pour construire la matrice de design ;
- `ordinary_least_squares` pour estimer les θ ;
- `solve_poly_reg` pour ajuster tous les modèles de degré 0 à 5 ;
- `plot_MSE_poly_fits` pour visualiser les MSE d'entraînement et de test.

1. Ajustement des modèles En utilisant `solve_poly_reg`, ajuster les modèles de degré 0 à 5 sur les données d'entraînement.

In [2]:
```
theta_hats = solve_poly_reg(x_train, y_train, max_order=5)
```

2. Calcul des MSE d'entraînement et de test Compléter la fonction `compute_mse` ci-dessous en utilisant `evaluate_poly_reg`.

In [3]:
```
compute_mse(x_train, x_test, y_train, y_test, theta_hats, max_order)
```

Lancer ensuite la visualisation des erreurs :

In [4]:
```
mse_train, mse_test = compute_mse(x_train, x_test, y_train, y_test, theta_hats, 5)
plot_MSE_poly_fits(mse_train, mse_test, 5)
```

Credit : Neuromatch Academy
Notebook : Notebook1

Exercice 5 (Validation croisée pour la sélection de modèle)
On souhaite comparer plusieurs modèles de régression polynomiale (de degré 0 à 5) en utilisant la **validation croisée** k**-fold**. On rappelle que l'on dispose d'un jeu d'entraînement

$$(x_{\text{train}}, y_{\text{train}})$$

généré selon le modèle bruité

$$y = x^2 - x - 2 + \varepsilon,$$

et que les fonctions `make_design_matrix`, `ordinary_least_squares`, `solve_poly_reg` et `evaluate_poly_reg` sont déjà fournies.

1. Mise en place de la validation croisée On souhaite effectuer une validation croisée à $k = 10$ folds pour comparer les modèles de degré 0 à 5. Compléter la fonction `cross_validate` ci-dessous afin de :

- diviser les données en k sous-ensembles avec `KFold`,
- entraîner les modèles sur les folds $k - 1$,
- évaluer leur MSE sur le fold de validation,
- stocker pour chaque degré le MSE obtenu sur chaque split.

In [5]:
```
cross_validate(x_train, y_train, max_order, n_splits)
```

2. Exécuter la validation croisée

In [6]:
```
max_order = 5
n_splits = 10
mse_all = cross_validate(x_train, y_train, max_order, n_splits)
plot_cross_validate_MSE(mse_all)
```

3. Analyse des résultats En observant les boxplots de MSE validation :

1. Quel degré de polynôme minimise en moyenne le MSE de validation ?
2. Comment les modèles de faible degré se comparent-ils aux modèles plus complexes en termes de **biais** et de **variance** ?
3. Pourquoi la validation croisée constitue-t-elle une meilleure méthode de sélection de modèle que la seule erreur d'entraînement ?

Credit: Neuromatch Academy
Notebook: Notebook2

Exercice 6
On souhaite obtenir des partitions d'apprentissage et de test afin de construire des estimateurs de risque. Dans cet exercice, on utilisera les fonctions suivantes du module `scikit-learn` :

- `train_test_split` pour séparer les données en deux (validation hold-out),
- `KFold` pour réaliser une validation croisée en blocs.

On générera des données simulées en deux dimensions avec Python comme suit :

In [7]:
```
from sklearn.datasets import make_classification
import numpy as np

X, y = make_classification(
        n_samples=500, n_features=2,
        n_informative=2, n_redundant=0,
        random_state=12345)
```

1. À l'aide de `train_test_split`, séparer les données en deux échantillons apprentissage/test avec 2/3 des données dans l'échantillon d'apprentissage.
2. Afficher l'index des individus de l'échantillon d'apprentissage. Faire de même pour l'échantillon test.
3. À l'aide de `KFold`, séparer les données en 10 blocs.
4. Afficher les index des individus qui se trouvent dans le bloc 4.

Linéarité en paramètres

Dans notre contexte, lorsque l'on dit qu'un modèle est **linéaire**, cela signifie qu'il est linéaire *par rapport à ses paramètres* (et non nécessairement par rapport aux variables d'entrée).

On rappelle qu'une application $g : \mathbb{R}^p \to \mathbb{R}$ est dite **linéaire** si elle préserve les combinaisons linéaires : pour tous $\theta, \theta' \in \mathbb{R}^p$ et pour tous scalaires $\alpha, \beta \in \mathbb{R}$,

$$g(\alpha\theta + \beta\theta') = \alpha g(\theta) + \beta g(\theta').$$

Ainsi, un modèle du type

$$f(x) = w_1\phi_1(x) + w_2\phi_2(x) + \cdots + w_p\phi_p(x) + b$$

est linéaire en ses paramètres $(w_1, \ldots, w_p, b)$, même si les fonctions $\phi_i(x)$ peuvent être non linéaires en x.

Exercice 7
Considérons le modèle

$$f(x) = w_1 x + w_2 x^2 + w_3 \sin(x) + b.$$

1. Le modèle est-il linéaire en *paramètres* ?
2. Le modèle est-il linéaire en *variables* ? Justifiez

Exercice 8 (Reconnaissance de visages avec SVM)
Dans cet exercice, on cherche à appliquer les SVM à un problème de **reconnaissance de visages**. On utilisera pour cela le dataset `Labeled Faces in the Wild (LFW)` proposé dans `scikit-learn`.

1. Chargement des données
On utilise uniquement les personnes ayant au moins 60 images afin de simplifier le problème de classification.

In [8]:

```python
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_lfw_people

lfw_people = fetch_lfw_people(min_faces_per_person=60)

print(lfw_people.DESCR)
```

1. Combien d'individus sont présents dans le dataset ?
2. Quelle est la dimension d'une image dans `lfw_people.images` ?
3. Que représente la matrice `lfw_people.data` ?

2. Visualisation de quelques visages

In [9]:

```python
target_names = lfw_people.target_names
fig, axes = plt.subplots(3, 5, figsize=(15, 8))
fig.suptitle("Some Faces in the Dataset", fontsize=16)

for i, ax in enumerate(axes.flat):
    ax.imshow(lfw_people.images[i], cmap='bone')
    ax.set_title(target_names[lfw_people.target[i]])

plt.tight_layout()
plt.show()
```

3. Séparation entraînement / test

In [10]:

```python
from sklearn.model_selection import train_test_split

X = lfw_people.data
y = lfw_people.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=3
)
```

4. Entraînement d'une SVM « vanilla »

In [11]:
```python
import warnings
warnings.filterwarnings("ignore")

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

clf = SVC()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print(f"Vanilla SVM Accuracy: {accuracy_score(y_test, y_pred)}")
```

5. Entraînement d'une SVM linéaire

In [12]:
```python
from sklearn.svm import LinearSVC

linear_svc = LinearSVC()
linear_svc.fit(X_train, y_train)
y_pred_linear = linear_svc.predict(X_test)

print(f"LinearSVC Accuracy: {accuracy_score(y_test, y_pred_linear)}")
```

Questions

1. Comparer les performances de `SVC()` et `LinearSVC()`. Pourquoi observe-t-on une différence de performance ?
2. Proposer deux améliorations possibles pour de meilleures performances.

Rappel (Le choix de C)

$\rightarrow$ régule le compromis biais/variance de la SVM :

- $\rightarrow$ Lorsque $C \downarrow$: la marge est privilégiée et les $\xi_i \uparrow$ $\Rightarrow$ beaucoup d'observations dans la marge ou mal classées (et donc beaucoup de vecteurs supports).
- $\rightarrow$ Lorsque $C \uparrow$: les $\xi_i \downarrow$ donc moins d'observations mal classées $\Rightarrow$ meilleur ajustement mais petite marge $\Rightarrow$ risque de surajustement.